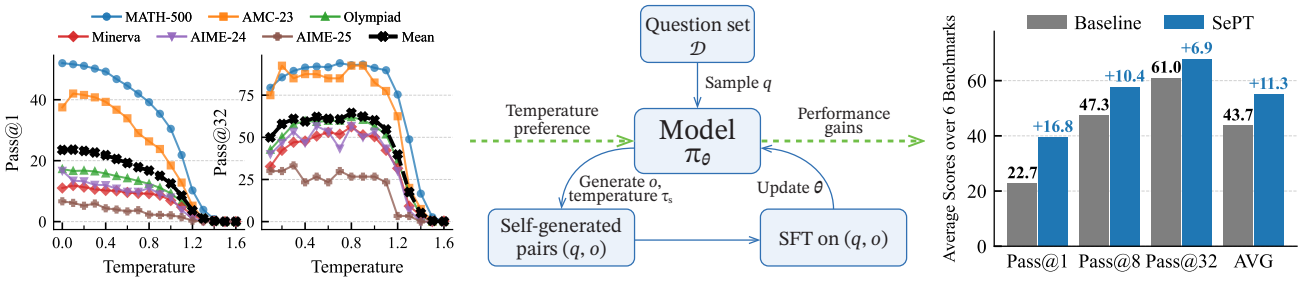
A Model Can Help Itself: Reward-Free Self-Training for LLM Reasoning

Mengqi Li[1] Lei Zhao[2] Anthony Man-Cho So[3] Ruoyu Sun[1,†] Xiao Li[1,†]
mengqili1@link.cuhk.edu.cn, l.zhao@sjtu.edu.cn,
manchoso@se.cuhk.edu.hk, {sunruoyu,lixiao}@cuhk.edu.cn

[1]The Chinese University of Hong Kong, Shenzhen [2]Shanghai Jiao Tong University
[3]The Chinese University of Hong Kong
[†]Corresponding authors

Abstract Can language models improve their reasoning performance without external rewards, using only their own sampled responses for training? We show that they can. We propose Self-evolving Post-Training (SePT), a simple post-training method that alternates between self-generation and training on self-generated responses. It repeatedly samples questions, uses the model itself to generate responses under a specified sampling temperature, and then trains the model on the self-generated data. In this self-training loop, we use an online data refresh mechanism, where each new batch is generated by the most recently updated model. Across six math reasoning benchmarks, SePT improves a strong no-training baseline, defined as the untuned base model evaluated at its best swept decoding temperature, on several tested models. Additional ablations demonstrate the importance of online data refresh and temperature dynamics. Overall, our results identify a practical regime where reasoning can be improved using self-generated supervision alone.

Code: https://github.com/ElementQi/SePT



(a) Base model Pass@1 and Pass@32 across evaluation temperatures.

(b) Workflow of SePT.

(c) Mean Pass@1, 8, 32, and AVG for Baseline and SePT.

Figure 1 Motivation, workflow, and main empirical result of Self-evolving Post-Training (SePT) on Qwen2.5-Math-7B. **(a)** Pass@1 and Pass@32 of the untuned base model under different evaluation temperatures τ_{eval}. Pass@1 peaks at low temperatures, while Pass@32 is higher at more moderate temperatures, indicating that different temperatures favor different parts of the model's reasoning distribution. **(b)** The SePT training loop. At each iteration, the current model samples responses at a specified sampling temperature τ_s, forms self-generated pairs (q, o), and is then updated by standard SFT training on these pairs. The updated model is used to generate the next batch of responses. **(c)** Mean Pass@1, Pass@8, Pass@32, and their unweighted AVG over six math reasoning benchmarks for the strong no-training baseline and SePT. The no-training baseline is the untuned base model evaluated at its best swept evaluation temperature τ_{eval}. SePT achieves strong gains over the baseline, without rewards, verifiers, or teacher-provided solutions.

1 Introduction

Large language models (LLMs) [1–3] have achieved good performance in many machine learning tasks [4]. Recently, there has been a surge of interest in training LLMs with Chain-of-Thoughts (CoT) [5] reasoning paths for complex mathematical tasks, as demonstrated by models like OpenAI-o1 and DeepSeek-R1 [6, 7]. Consequently, developing efficient training strategies for reasoning models has attracted growing interest.

Background and Related Work. To improve LLM reasoning performance, reinforcement learning with verifiable reward (RLVR) has become a popular approach [7]. RLVR leverages verifiable, rule-based rewards (e.g., ground-truth answers to math questions) to provide a training signal, guiding the model to move its policy towards trajectories that yield correct outcomes. Common RL algorithms for reasoning include GRPO [8], PPO [9], ReMax [10], DAPO [11], Dr. GRPO [12], GSPO [13], etc. Many recent works, e.g., [11, 12, 14–21], have further sought to explore the potential of RL in complex reasoning tasks.

Beyond RL, a standard alternative is SFT on reasoning traces distilled from stronger reasoning models such as DeepSeek-R1. In particular, [22] studies a simple recipe for reasoning gains together with test-time scaling, [23] argues that a relatively small amount of carefully chosen reasoning data can already be highly effective, [24] emphasizes that the structural form of demonstrations can matter more than their exact content, and [25] combines curriculum SFT with later preference optimization and RL stages for long-CoT training. The recent work [26] complements this line by systematically studying reasoning data recipes for SFT, including questions of data composition, filtering, and curation, which leads to the OpenThoughts dataset. More related works can be found in Appendix A.

It has been shown that RLVR and SFT can improve reasoning performance when a model's own sampled responses are paired with an external reward, and when teacher-provided solution traces are available, respectively. At the same time, the temperature-sensitivity evidence in Figure 1a suggests that untuned base models may already contain a useful intrinsic ordering over candidate reasoning paths. This raises the possibility that self-generated samples, together with additional post-training compute, may be able to improve reasoning performance. We therefore ask the following research question:

Can LLMs improve their reasoning performance without external rewards or teacher solutions, using only their own sampled responses for training?

Main Contributions. In this work, we provide an affirmative answer to this question by introducing Self-evolving Post-Training (SePT), a reward-free self-training paradigm for LLM reasoning. Our main contributions are summarized below.

(C.1) We propose Self-evolving Post-Training (**SePT**), a simple reward-free post-training method that alternates between self-generation of responses and standard training on the self-generated traces. In its default form, SePT uses a single rollout per prompt.

(C.2) SePT features flexible choices of the sampling temperature during self-generation. We provide an analysis to discuss the effects of different regimes of sampling temperatures, justifying the temperature setup used in our main experiments.

(C.3) We present empirical evidence across six math reasoning benchmarks showing that SePT can improve over a strong *no-training baseline*, defined as the untuned base model evaluated at the best swept decoding temperature. These gains are obtained without rewards, verifiers, or teacher signals, with the clearest improvements appearing on untuned Qwen family models. In some settings, SePT can even approach the performance of models trained by RLVR. Additional ablations further clarify the importance of online data refresh and temperature dynamics in SePT.

2 The SePT Method

Throughout this paper, we will use π_θ with parameters θ to denote an LLM. Given a prompt q, it generates an output sequence $o = (o_1, o_2, \ldots, o_T)$ through the conditional probability distribution $\pi_\theta(o \mid q)$ in an autoregressive manner, namely, $\pi_\theta(o \mid q) = \prod_{k=1}^{T} \pi_\theta(o_k \mid q, o_{<k})$, where $o_{<k}$ denotes the tokens preceding o_k. For the background on SFT and GRPO formulations, we refer to Appendix G.

Algorithm 1 Iterative Self-evolving Post-Training (SePT)

Input initial model $\pi_{\theta_{\text{init}}}$; task prompts $\mathcal{D}$; hyperparameters: sampling temperature τ_s, training temperature τ_t, rollouts per prompt G, number of refresh rounds M, optimization steps per round μ.

model initialization $\pi_\theta \leftarrow \pi_{\theta_{\text{init}}}$
for step $= 1, \ldots,$ M **do**
  $\pi_{\text{old}} \leftarrow \pi_\theta$
  Sample a batch of questions $\mathcal{D}_b$ from $\mathcal{D}$
  Initialize an empty set for the training batch: $\mathcal{D}_{\text{SePT}} \leftarrow \varnothing$
  for each question $q \in \mathcal{D}_b$ **do**
    Sample G outputs $\{o_i\}_{i=1}^{G} \sim \pi_{\text{old}}(\cdot \mid q, \tau_s)$
    Add the generated pairs $\{(q, o_i)\}_{i=1}^{G}$ to $\mathcal{D}_{\text{SePT}}$
  for SePT iteration $= 1, \ldots, \mu$ **do**
    Update the model π_θ by minimizing the SePT loss (Equation (1)) on $\mathcal{D}_{\text{SePT}}$.

Output π_θ

2.1 Algorithmic Procedure of SePT

Figure 1a shows a clear temperature-sensitive performance for the untuned Qwen2.5-Math-7B model: Low decoding temperatures maximize Pass@1, whereas moderate temperatures are preferable for Pass@32. This suggests that useful reasoning paths may already exist in the untuned base models, while different temperatures expose different parts of its reasoning distribution. Motivated by this observation, we propose Self-evolving Post-Training (SePT) for LLM reasoning. It is a reward-free online self-training method. The core loop, illustrated in Algorithm 1, involves two steps:

(S.1) **Self-generation of responses:** Using the model itself to sample outputs with a sampling temperature τ_s.

(S.2) **SFT:** The model is then updated by performing standard SFT training on these self-generated data.

We define the SePT loss with distinct sampling and training temperatures:

$$\mathcal{L}_{\text{SePT}} = -\mathbb{E}_{q\sim\mathcal{D}, o\sim\pi_{\text{old}}(\cdot|q;\tau_s)} \left[\log \pi_\theta(o \mid q; \tau_t)\right] \tag{1}$$

Here, o denotes an autoregressive output sequence, and $\log \pi_\theta(o \mid q; \tau_t)$ is the sum of token-level log-probabilities computed with logits scaled by τ_t. While in principle τ_t can be tuned, we find that the standard SFT setting of $\tau_t = 1$ is sufficient for stable learning, which we adopt in our main experiments.

Although SePT invites comparison with RLVR, the update mechanisms are fundamentally different. RLVR relies on reward-weighted policy optimization. SePT, by contrast, minimizes a plain negative log-likelihood objective on self-generated data.

2.2 Discussion on Temperature Dynamics in SePT

We now analyze the interplay between τ_s and τ_t (with $\tau_t = 1$) to discuss the temperature dynamics in SePT.

Let $\zeta = (q, o_{<k})$ denote a prefix state and $d_{\text{old}}(\zeta)$ be the expected number of visits to prefix ζ when sampling $q \sim \mathcal{D}$ and $o \sim \pi_{\text{old}}(\cdot \mid q; \tau_s)$. Below, we provide a chain-rule decomposition of the sequence cross-entropy loss in SePT and then provide a theoretical interpretation for its design principle.

Proposition 1 (sequence-level KL decomposition of SePT)**.** *Within one SePT round, for C_b independent of θ, we have*

$$\mathcal{L}_{\text{SePT}}(\theta) = \mathbb{E}_{q,o}\left[\sum_{k=1}^{|o|} -\log \pi_\theta(o_k \mid q, o_{<k}; \tau_t)\right] = C_b + \sum_{\zeta} d_{\text{old}}(\zeta)\ \text{KL}\big(\pi_{\text{old}}(\cdot \mid \zeta; \tau_s) \,\|\, \pi_\theta(\cdot \mid \zeta; \tau_t)\big).$$

This gives a sequence-level characterization of the round objective and indicates that SePT is an occupancy-weighted forward-KL projection onto the current τ_s-temperature self-teacher's distribution.

Theorem 1 (temperature-ratio logit scaling)**.** *For any prefix ζ with $d_{\text{old}}(\zeta) > 0$, the pointwise optimum of the loss $\mathcal{L}_{\text{SePT}}$ in Proposition 1 satisfies $p^*_\theta(\cdot \mid \zeta) = \pi_{\text{old}}(\cdot \mid \zeta; \tau_s)$. Equivalently, once written $\pi_{\text{old}}(\cdot \mid \zeta; \tau_s) =$*

$\mathrm{softmax}\left(\frac{z_{\mathrm{old}}(\zeta)}{\tau_s}\right)$, $p^*_\theta(\cdot \mid \zeta) = \mathrm{softmax}\left(\frac{z^*(\zeta)}{\tau_t}\right)$, *there exists a scalar* $c(\zeta) \in \mathbb{R}$ *such that* $z^*(\zeta) = \frac{\tau_t}{\tau_s} z_{\mathrm{old}}(\zeta) + c(\zeta)\mathbf{1}$. *Hence, for any two tokens* i, j,

$$z^*_i(\zeta) - z^*_j(\zeta) = \frac{\tau_t}{\tau_s}\big(z^{\mathrm{old}}_i(\zeta) - z^{\mathrm{old}}_j(\zeta)\big).$$

In particular, when $\tau_s < \tau_t$, *every pairwise logit margin is amplified by the factor* $\tau_t/\tau_s > 1$.

Discussion on Temperature Dynamics. Theorem 1 partly characterizes the effect of SePT through the temperature ratio τ_t/τ_s. The regime $\tau_s < \tau_t$ yields margin amplification and is therefore the principled regime for top-modes improvement. The coupled case $\tau_s = \tau_t$ is neutral in population and has no specific first-order updating direction at the current model (see Appendix F.1). The regime $\tau_s > \tau_t$ induces margin compression and may be useful for objectives that favor diversity or smoothing, although it is not the target regime of this paper.

This result gives an interpretation of the SePT design, where we chose $\tau_s < \tau_t$ as the main training stage. Namely, sampling at a lower temperature (i.e., $\tau_s < \tau_t = 1$) preserves the local token ordering (sampled from the teacher), while amplifying the model's existing logit margins. This is further illustrated by the trajectory-level case study in Section 3.2.1.

We note that this theorem characterizes the ideal local target induced by one SePT round. Since a language model uses the same parameters θ for all prefixes, training seeks to fit these prefix-wise targets jointly rather than optimizing each conditional distribution independently. Additionally, although the analysis is stated for a fixed τ_s within each SePT round, it directly applies to the annealed schedule used in our experiments, as in each round τ_s is fixed during self-generation and training and only changes across rounds. The proofs of Proposition 1 and Theorem 1 are put in Appendix F.2.

3 Experiments

3.1 Experiment Setup

We conduct all experiments using the verl framework [27]. In the following, we provide core experiment settings, while providing comprehensive training and evaluation configurations in Appendix B.

Datasets. Our primary training question set is DeepScaleR (DSR) [16]. We also use OpenThoughts-Math (OTM) [26] as an alternative training question set. We evaluate all models on a suite of six math reasoning benchmarks: MATH-500 [28], AMC-23 [29], Minerva Math (Minerva) [30], OlympiadBench (Olympiad) [31], AIME-24 [29], and AIME-25 [29]. We will report the mean score of the six benchmarks in the main context, while providing the detailed score of each benchmark in Appendix C for each experiment.

Hyperparameters. For SePT, we use low-temperature (i.e., low-τ_s regime) self-generation as the main training stage and the default rollout count is $G = 1$. For GRPO, we use the standard multi-rollout setting in our implementation. Most hyperparameters for both algorithms are kept as the default ones in the verl framework.

Baseline. Unless otherwise stated, *base model* refers to a model not trained by our considered methods, while *baseline* denotes a no-training reference obtained by sweeping τ_{eval} for the base model and selecting the single temperature that maximizes AVG, where AVG is the unweighted average of the six-benchmark means of Pass@1, Pass@8, and Pass@32. All reported Pass@k values in the "Baseline" row are computed using this same selected temperature. This is a strong no-training baseline since the base model demonstrates decoding temperature sensitivity as shown in Figure 1a. In contrast, trained models (by SePT and GRPO) are reported at $\tau_{\mathrm{eval}} = 1$, without post-hoc decoding temperature tuning, in order to clearly illustrate the effect of training.

3.2 SePT for LLM Reasoning: Analysis and Performance

3.2.1 A Trajectory-Level Case Study

We present an illustrative case study on a problem from the MATH-500 dataset using the base Qwen2.5-Math-7B model, π_θ, and its SePT-trained version, $\pi_{\hat{\theta}}$. As shown in Figure 2, the base model fails on this problem across eight sampled attempts, and the figure visualizes one representative failed trajectory $[A, B]$. In contrast, the SePT model produces the correct trajectory $[\hat{A}, \hat{B}]$ in the displayed run. We note that A and $\hat{A}$ (and likewise B and $\hat{B}$) are semantically similar, so the contrast is between competing reasoning branches of similar semantic form.

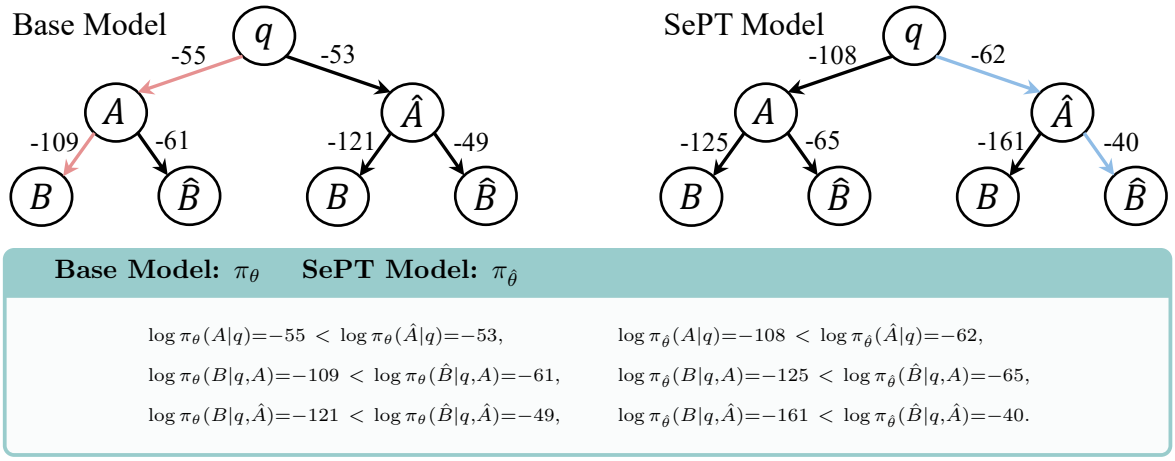


Figure 2 Trajectory-level probability analysis on a representative MATH-500 example for the base model π_θ and the SePT-trained model $\pi_{\hat{\theta}}$, where all displayed trajectories are generated with decoding temperature $\tau_{\text{eval}} = 1$. The base model fails on this problem across eight sampled attempts; we visualize one representative failed trajectory $[A, B]$, where B is the **wrong** response (highlighted in light red). The SePT model produces the correct trajectory $[\hat{A}, \hat{B}]$, where $\hat{B}$ contains the **correct** response (highlighted in light blue). The displayed values are the log-probabilities assigned by each model to the corresponding prefixes and suffixes. SePT substantially widens the probability margins in favor of the superior reasoning branch, primarily by downweighting competing paths. The full question and responses are provided in Appendix E.

This example suggests that the base model may already contain a useful preference ordering over candidate reasoning steps, but the corresponding margins are small in order to reliably keep generation on the superior branch. At the first branching point, the model assigns slightly higher probability to the alternative prefix $\hat{A}$ than to the sampled prefix A ($\log \pi_\theta(\hat{A} \mid q) = -53$ versus $\log \pi_\theta(A \mid q) = -55$), yet the gap is only 2. Once the model commits to prefix A, it still assigns substantially higher probability to the semantically better continuation $\hat{B}$ than to the sampled continuation B (-61 versus -109). Moreover, under the alternative prefix $\hat{A}$, the model strongly prefers $\hat{B}$ over B (-49 versus -121). Taken together, these scores indicate that the correct reasoning branch may already exist in the model's reasoning distribution, while the model does not reliably follow it during sampling.

SePT changes this example primarily by widening these relative preference margins. After training, the prefix gap in favor of $\hat{A}$ increases from 2 to 46. Along the branch starting with $\hat{A}$, the suffix gap in favor of $\hat{B}$ increases from 72 to 121. At the trajectory level, it is interesting to observe that the absolute log-probability of the correct full path $[\hat{A}, \hat{B}]$ remains unchanged in this example before and after SePT, i.e., $-53 - 49 = -102$ for the base model and $-62 - 40 = -102$ for the SePT model. What changes dramatically after SePT is the suppression of the competing wrong path $[A, B]$, whose score drops from -164 to -233, widening the margin between $[A, B]$ and $[\hat{A}, \hat{B}]$ from 62 to 131.

Figure 2 is a trajectory-level illustration at fixed decoding temperature $\tau_{\text{eval}} = 1$. Lowering τ_{eval} only changes the untuned base model's existing output distribution at inference time, whereas SePT updates the model parameters by training on the whole self-generated trajectories. As a result, SePT can modify the conditional distributions themselves and enlarge preference margins across many prompts and intermediate states. The fact that SePT can surpass the no-training base model's best swept decoding baseline therefore indicates an improved training effect, rather than merely a decoding tuning effect on the next token.

3.2.2 SePT Improves Over the No-Training Baseline

In Table 1, we report the mean benchmark performance, averaged over the six math benchmarks. For a compact comparison, we define AVG as the unweighted average of Pass@1, Pass@8, and Pass@32.

On Qwen2.5-Math-7B, SePT improves Pass@1, Pass@8, and Pass@32 of the baseline from 22.7/47.3/61.0 to 39.5/57.7/67.9, increasing AVG from 43.7 to 55.0. On Qwen2.5-7B, SePT again improves all three metrics, raising AVG from 44.1 to 50.7. The remaining two instruction-tuned models, i.e., Qwen2.5-7B-Instruct and DeepSeek-Math-7B-Instruct, show smaller improvements. Their AVG gains are 0.5 and 1.2, respectively. The smaller gains of these models are partly expected as shown in their decoding temperature sweeps in Appendix C.1.

Table 1 Mean benchmark performance (average over six math benchmarks) on the four models for which SePT improves AVG over the no-training baseline. AVG is the unweighted average of Pass@1, Pass@8, and Pass@32. Here, Base Model refers to a model not trained by our considered methods and evaluated at the decoding temperature $\tau_{\text{eval}} = 1$, while Baseline is this untuned base model evaluated at its best swept decoding temperature. Values in parentheses denote the change relative to Baseline.

Model	Method	Mean Benchmark Pass@k			
		$k = 1$	$k = 8$	$k = 32$	**AVG**
Qwen2.5-Math-7B	Base Model	12.5	42.0	60.1	38.2
	Baseline	22.7	47.3	61.0	43.7
	SePT	39.5 (+16.8)	57.7 (+10.4)	67.9 (+6.9)	55.0 (+11.3)
Qwen2.5-7B	Base Model	10.7	37.5	55.4	34.6
	Baseline	21.3	48.7	62.2	44.1
	SePT	32.3 (+11.0)	54.6 (+5.9)	65.3 (+3.1)	50.7 (+6.6)
Qwen2.5-7B-Instruct	Base Model	35.7	55.5	63.6	51.6
	Baseline	36.8	56.6	67.3	53.6
	SePT	36.6 (-0.2)	56.8 (+0.2)	69.0 (+1.7)	54.1 (+0.5)
DeepSeek-Math-7B-Instruct	Base Model	13.6	33.0	47.0	31.2
	Baseline	15.4	34.6	48.1	32.7
	SePT	15.9 (+0.5)	35.5 (+0.9)	50.3 (+2.2)	33.9 (+1.2)

Table 2 Mean benchmark comparison between SePT and RLVR (GRPO) on Qwen2.5-Math-7B and DeepSeek-Math-7B-Instruct using training question sets DeepScaleR (DSR) and OpenThoughts-Math (OTM). The percentage next to each training set is the share of evaluation questions matched by the benchmark contamination audit. Values in parentheses indicate the change of AVG by switching from DSR to OTM.

Model	Method	**DSR** (Bench. Contam.: 5.43%)				**OTM** (Bench. Contam.: 0.19%)			
		$k = 1$	$k = 8$	$k = 32$	**AVG**	$k = 1$	$k = 8$	$k = 32$	**AVG**
Qwen2.5-Math-7B	SePT	39.5	57.7	67.9	55.0	40.8	58.6	66.2	55.2 (+0.2)
	GRPO	43.8	61.8	71.6	59.1	39.5	59.0	71.4	56.6 (-2.5)
DeepSeek-Math-7B-Instruct	SePT	15.9	35.5	50.3	33.9	15.8	35.1	48.2	33.0 (-0.9)
	GRPO	18.2	37.7	51.0	35.6	16.9	35.1	48.2	33.4 (-2.2)

In particular, both Qwen2.5-7B-Instruct and DeepSeek-Math-7B-Instruct do not show clear improvements by sweeping decoding temperature in interval $\tau_{\text{eval}} \leq 1$. Hence, SePT does not significantly improve the overall Pass@k for the two instruction-tuned models. We note that these are only qualitative observations on which models SePT might improve over the baseline significantly, whereas a quantitative analysis is left for future exploration. Let us also mention that even RLVR (GRPO) also just modestly improves over Baseline in this case; see the DeepSeek-Math-7B-Instruct row in Table 2 in the next subsection.

These empirical results show that our simple self-training method can surpass the strong no-training baseline. Let us remark that the SePT-trained models are evaluated at $\tau_{\text{eval}} = 1$ without post-hoc decoding temperature tuning, whereas Baseline is evaluated at the best swept τ_{eval} for the no-training base model, in order to clearly illustrate whether the effect of self-training in SePT can improve over an upper bound (in terms of decoding temperature) of the base model. For a comparison, we refer to the "Base Model" row in Table 1 where the no-training model is evaluated at $\tau_{\text{eval}} = 1$, and SePT improves over it more significantly. These observations illustrate that the improvement of SePT over the Baseline comes from our self-training paradigm and is beyond a post-hoc decoding tuning only.

3.2.3 Comparison with Other Training-based Methods

We next compare our self-training method SePT with RLVR (GRPO). We use both the primary training set DSR as well as OpenThoughts-Math (OTM) to train two models from different families, i.e., Qwen2.5-Math-7B and DeepSeek-Math-7B-Instruct. The two question sets have nearly the same size, so the difference primarily reflects data composition rather than data volume. The results are displayed in Table 2, which exhibits a clear asymmetry for GRPO and SePT. GRPO-trained models on the DSR training question set improve over SePT

Table 3 Average training compute per update for the DSR runs in Table 2 (out of 900 steps). The FLOPs accounting follows the standard dense-transformer estimate detailed in Appendix B.5.

Model	FLOPs / update (10^{15})		Ratio
	SePT	GRPO	GRPO / SePT
Qwen2.5-Math-7B	7.53	67.23	8.93×
DeepSeek-Math-7B-Instruct	4.55	46.92	10.32×

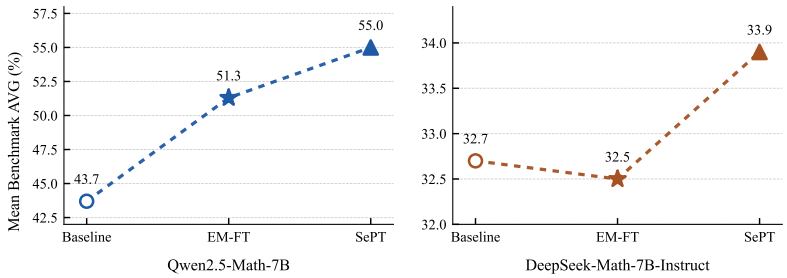


Figure 3 Comparison with EM-FT, a entropy-minimization method. SePT achieves a higher AVG on both Qwen2.5-Math-7B and DeepSeek-Math-7B-Instruct.

clearly. However, the advantage of GRPO over SePT significantly decreases when the training set is changed to OTM. In contrast, SePT is much less sensitive to this change of training set: On Qwen2.5-Math-7B, switching from DSR to OTM slightly increases AVG from 55.0 to 55.2, while on DeepSeek-Math-7B-Instruct, the decrease is just -0.9.

To further support our observations, we conduct a benchmark contamination check for the two training question sets, in order to quantify the ratio of benchmark questions (or their similar variants) that appear in the two sets. DSR has a clearly higher contamination ratio than OTM (5.43% vs. 0.19%), as shown in the column headers in Table 2. We give the full contamination audit in Appendix D. Additionally, we compare the training compute of the DSR runs in Table 3, GRPO uses 8.93× and 10.32× more average FLOPs than SePT on Qwen2.5-Math-7B and DeepSeek-Math-7B-Instruct, respectively. These results together indicate that in some settings, especially under the OTM question set, SePT can even approach the performance of the reward-based approach RLVR using much less compute, at least on the representative Qwen and DeepSeek math models.

Figure 3 compares SePT with EM-FT [32], a reward-free entropy minimization method. Empirically, SePT is stronger on both models: On Qwen2.5-Math-7B, AVG improves from 51.3 under EM-FT to 55.0, and on DeepSeek-Math-7B-Instruct, SePT reaches 33.9 while EM-FT is slightly below the baseline. Although both methods are reward-free, they optimize fundamentally different objectives. EM-FT samples rollouts from the current model and minimizes token-level entropy on the visited prefixes, whereas SePT applies standard cross-entropy training to the realized low-temperature trajectory itself. Thus, EM-FT reduces token-level entropy of the next-token distribution at each visited prefix, while SePT directly reinforces the realized sample trajectory. This can also be inferred from Proposition 1.

3.2.4 Test of General Domain Benchmark

A natural concern is that training on self-generated math traces may overspecialize the model and harm more general capabilities. To test this, we evaluate the untuned base model Qwen2.5-Math-7B together with its SePT- and GRPO-trained counterparts on a set of non-math benchmarks using the Language Model Evaluation Harness [33]. Specifically, we report results on IFEval [34], MMLU-Pro [35], BigBenchHard (BBH) [36], GPQA [37], and MuSR [38]. As shown in Table 4, neither SePT nor GRPO causes a noticeable drop in general ability. SePT is essentially neutral relative to the base model, and GRPO shows a similar pattern. This indicates that the reward-free self-training in SePT does not essentially degrade the model's general capabilities on the benchmarks considered here.

Table 4 General domain benchmark results for the untuned base model Qwen2.5-Math-7B and its SePT- and GRPO-trained counterparts.

Method	IFEval	BBH	GPQA	MuSR	MMLU-Pro	Avg.
Base	23.4	47.5	29.9	41.4	32.1	34.9
SePT	23.6	47.3	30.6	41.5	32.2	35.0
GRPO	24.6	47.7	30.1	41.4	32.3	35.2

Table 5 Comparison between SePT (Offline) and SePT on Qwen2.5-Math-7B under the same evaluation setup as in Table 1. Values in parentheses denote SePT (Offline)/SePT−Baseline.

Method	Mean Benchmark Pass@k			
	$k = 1$	$k = 8$	$k = 32$	**AVG**
Baseline	22.7	47.3	61.0	43.7
SePT (Offline)	21.0 (-1.7)	50.2 (+2.9)	65.2 (+4.2)	45.5 (+1.8)
SePT	39.5 (+16.8)	57.7 (+10.4)	67.9 (+6.9)	55.0 (+11.3)

3.3 Ablation Study

We now study which components of SePT are responsible for the observed gains. Unless otherwise stated, all ablation experiments in this section use Qwen2.5-Math-7B and DSR as the base model and training question set.

3.3.1 Effect of Online Data Refresh: Comparison with SePT (Offline)

To isolate the role of online data refresh, we construct an offline variant. We first use the untuned base model to generate a fixed self-training dataset with the same starting low sampling temperature as SePT, and then perform standard SFT on this frozen dataset. Table 5 reports the results under the same evaluation protocol as in Table 1.

SePT (Offline) still yields gains at larger k: Pass@8 increases from 47.3 to 50.2, and Pass@32 from 61.0 to 65.2. However, its Pass@1 drops from 22.7 to 21.0, so the overall improvement in AVG is limited to +1.8. By contrast, SePT improves all three metrics substantially, especially Pass@1, and raises AVG by +11.3. This comparison indicates that the online component is not a superficial implementation detail. Combining self-generation with parameter updates in an online manner appears important for turning self-generated supervision into large gains.

3.3.2 Effect of Temperature Dynamics

In Table 6, we compare different temperature choices ($\tau_s < \tau_t = 1$ and $\tau_s = \tau_t = 1$) in SePT. We also tried the regime $\tau_s > \tau_t = 1$ in preliminary runs. In our current math-training setup, this regime was unstable and led to severe degradation. Under the neutral setting ($\tau_s = \tau_t = 1$), Pass@1 falls below the no-training baseline (19.3 vs. 22.7), while Pass@8 and Pass@32 improve over it, yielding an AVG of 44.6. Under the low-τ_s setting ($\tau_s < \tau_t = 1$), all three metrics improve strongly. We therefore adopt the low-τ_s regime as the main training stage in our experiments. Nonetheless, this does not necessarily rule out the neutral regime. In preliminary experiments, applying a short neutral stage after the main low-τ_s training stage sometimes yielded a slightly higher Pass@32, which is also consistent with the high-k gains observed in Table 6.

3.4 Limitation

Despite the positive regime identified above, SePT may not be uniformly effective across model families. As a continuation of Table 1, Table 7 reports a representative case, Llama-3.1-8B-Instruct, where SePT is not beneficial. On Llama-3.1-8B-Instruct, SePT remains below the strong no-training baseline in AVG under both DSR and OTM. This indicates that our self-training can be model-dependent, and therefore we do not expect a single SePT recipe to transfer universally across all model families.

Let us also mention that the limitation case is indeed not unique to SePT. On Llama-3.1-8B-Instruct, GRPO is stronger than SePT under the DSR training question set, but this advantage does not persist under the alternative OTM question set. In this setting, GRPO also falls below the baseline.

Table 6 Comparison between different temperature dynamics on Qwen2.5-Math-7B. Values in parentheses denote Method−Baseline.

Method	Mean Benchmark Pass@k			
	$k = 1$	$k = 8$	$k = 32$	**AVG**
Baseline	22.7	47.3	61.0	43.7
$\tau_s = \tau_t$	19.3 (-3.4)	50.1 (+2.8)	64.3 (+3.3)	44.6 (+0.9)
$\tau_s < \tau_t$	39.5 (+16.8)	57.7 (+10.4)	67.9 (+6.9)	55.0 (+11.3)

Table 7 Limitation of SePT with mean-benchmark summaries using both the DSR and OTM training question sets. Values in parentheses show method−baseline. Base Model and Baseline are repeated under both DSR and OTM columns for easier comparison.

Model	Method	Mean Benchmark Pass@k (DSR)				Mean Benchmark Pass@k (OTM)			
		$k = 1$	$k = 8$	$k = 32$	**AVG**	$k = 1$	$k = 8$	$k = 32$	**AVG**
Llama-3.1-8B-Instruct	Base Model	14.6	36.2	50.8	33.9	14.6	36.2	50.8	33.9
	Baseline	20.5	40.9	55.3	38.9	20.5	40.9	55.3	38.9
	SePT	20.0	39.8	53.4	37.8 (-1.1)	20.1	40.4	54.1	38.2 (-0.7)
	GRPO	25.2	44.0	54.4	41.2 (+2.3)	19.7	39.9	50.9	36.8 (-2.1)

4 Conclusion

We studied whether a language model can improve its reasoning performance using only its self-generated responses, without rewards, verifiers, or teacher-provided solution traces. To this end, we proposed Self-evolving Post-Training (SePT), a simple reward-free online self-training method that alternates between self-generation and standard training on the resulting self-generated traces. We provided an analysis of the temperature dynamics in SePT. Empirically, SePT improves over a strong no-training baseline on several tested models. We also compared our self-training method SePT with the reward-based RLVR approach GRPO. In some settings, SePT can even approach the performance of GRPO, using much less compute without rewards. We also demonstrated that replacing online refresh with a frozen offline self-generated dataset leads to much weaker gains, indicating that the online component is important.

Overall, our results identify a concrete regime in which mathematical reasoning can be improved through combining self-generated supervision and standard SFT training. A natural next step is to understand more precisely which model properties make SePT effective, when it fails, and whether the same self-improvement behavior extends beyond mathematical reasoning to broader reasoning domains.

Broader Impacts. This work proposes a simple self-training approach for LLM reasoning. Its main impact is technical, potentially improving the efficiency of reasoning post-training. We do not anticipate direct negative societal impacts beyond those generally associated with stronger LLMs.

References

[1] Josh Achiam, Steven Adler, Sandhini Agarwal, Lama Ahmad, Ilge Akkaya, Florencia Leoni Aleman, Diogo Almeida, Janko Altenschmidt, Sam Altman, Shyamal Anadkat, et al. GPT-4 technical report. *arXiv preprint arXiv:2303.08774*, 2023.

[2] Abhimanyu Dubey, Abhinav Jauhri, Abhinav Pandey, Abhishek Kadian, Ahmad Al-Dahle, Aiesha Letman, Akhil Mathur, Alan Schelten, Amy Yang, Angela Fan, et al. The llama 3 herd of models. *arXiv preprint arXiv:2407.21783*, 2024.

[3] An Yang, Baosong Yang, Beichen Zhang, Binyuan Hui, Bo Zheng, Bowen Yu, Chengyuan Li, Dayiheng Liu, Fei Huang, Haoran Wei, et al. Qwen2. 5 technical report. *arXiv preprint arXiv:2412.15115*, 2024.

[4] Sébastien Bubeck, Varun Chandrasekaran, Ronen Eldan, Johannes Gehrke, Eric Horvitz, Ece Kamar, Peter Lee, Yin Tat Lee, Yuanzhi Li, Scott Lundberg, et al. Sparks of artificial general intelligence: Early experiments with GPT-4. *arXiv preprint arXiv:2303.12712*, 2023.

[5] Jason Wei, Xuezhi Wang, Dale Schuurmans, Maarten Bosma, brian ichter, Fei Xia, Ed Chi, Quoc V Le, and Denny Zhou. Chain-of-thought prompting elicits reasoning in large language models. *Advances in neural information processing systems*, 35:24824–24837, 2022.

[6] Aaron Jaech, Adam Kalai, Adam Lerer, Adam Richardson, Ahmed El-Kishky, Aiden Low, Alec Helyar, Aleksander Madry, Alex Beutel, Alex Carney, et al. Openai o1 system card. *arXiv preprint arXiv:2412.16720*, 2024.

[7] Daya Guo, Dejian Yang, Haowei Zhang, Junxiao Song, Peiyi Wang, Qihao Zhu, Runxin Xu, Ruoyu Zhang, Shirong Ma, Xiao Bi, et al. Deepseek-r1 incentivizes reasoning in llms through reinforcement learning. *Nature*, 645(8081): 633–638, 2025.

[8] Zhihong Shao, Peiyi Wang, Qihao Zhu, Runxin Xu, Junxiao Song, Xiao Bi, Haowei Zhang, Mingchuan Zhang, Y. K. Li, Y. Wu, and Daya Guo. Deepseekmath: Pushing the limits of mathematical reasoning in open language models. *arXiv preprint arXiv:2402.03300*, 2024.

[9] John Schulman, Filip Wolski, Prafulla Dhariwal, Alec Radford, and Oleg Klimov. Proximal policy optimization algorithms. *arXiv preprint arXiv:1707.06347*, 2017.

[10] Ziniu Li, Tian Xu, Yushun Zhang, Zhihang Lin, Yang Yu, Ruoyu Sun, and Zhi-Quan Luo. ReMax: A simple, effective, and efficient reinforcement learning method for aligning large language models. In *Proceedings of the 41st International Conference on Machine Learning (ICML)*, 2024.

[11] Qiying Yu, Zheng Zhang, Ruofei Zhu, Yufeng Yuan, Xiaochen Zuo, Yu Yue, Tiantian Fan, Gaohong Liu, Lingjun Liu, Xin Liu, et al. Dapo: An open-source llm reinforcement learning system at scale. *arXiv preprint arXiv:2503.14476*, 2025.

[12] Zichen Liu, Changyu Chen, Wenjun Li, Penghui Qi, Tianyu Pang, Chao Du, Wee Sun Lee, and Min Lin. Understanding r1-zero-like training: A critical perspective. *arXiv preprint arXiv:2503.20783*, 2025.

[13] Chujie Zheng, Shixuan Liu, Mingze Li, Xiong-Hui Chen, Bowen Yu, Chang Gao, Kai Dang, Yuqiong Liu, Rui Men, An Yang, et al. Group sequence policy optimization. *arXiv preprint arXiv:2507.18071*, 2025.

[14] Hugging Face. Open R1: A fully open reproduction of deepseek-r1, January 2025. URL https://github.com/huggingface/open-r1.

[15] Jiayi Pan, Junjie Zhang, Xingyao Wang, Lifan Yuan, Hao Peng, and Alane Suhr. Tinyzero, 2025. URL https://github.com/Jiayi-Pan/TinyZero. Accessed: 2025-01-24.

[16] Michael Luo, Sijun Tan, Justin Wong, Xiaoxiang Shi, William Y. Tang, Manan Roongta, Colin Cai, Jeffrey Luo, Li Erran Li, Raluca Ada Popa, and Ion Stoica. Deepscaler: Surpassing o1-preview with a 1.5b model by scaling rl, 2025. URL https://pretty-radio-b75.notion.site/DeepScaleR-Surpassing-O1-Preview-with-a-1-5B-Model-by-Scaling-RL-19681902c1468005bed8ca303013a4e2. Notion Blog.

[17] Weihao Zeng, Yuzhen Huang, Qian Liu, Wei Liu, Keqing He, Zejun Ma, and Junxian He. Simplerl-zoo: Investigating and taming zero reinforcement learning for open base models in the wild. *arXiv preprint arXiv:2503.18892*, 2025.

[18] Jingcheng Hu, Yinmin Zhang, Qi Han, Daxin Jiang, Xiangyu Zhang, and Heung-Yeung Shum. Open-reasoner-zero: An open source approach to scaling up reinforcement learning on the base model. *arXiv preprint arXiv:2503.24290*, 2025.

[19] Yiping Wang, Qing Yang, Zhiyuan Zeng, Liliang Ren, Lucas Liu, Baolin Peng, Hao Cheng, Xuehai He, Kuan Wang, Jianfeng Gao, et al. Reinforcement learning for reasoning in large language models with one training example. *arXiv preprint arXiv:2504.20571*, 2025.

[20] Ziniu Li, Congliang Chen, Tianyun Yang, Tian Ding, Ruoyu Sun, Ge Zhang, Wenhao Huang, and Zhi-Quan Luo. Knapsack RL: Unlocking exploration of LLMs via optimizing budget allocation. *arXiv preprint arXiv:2509.25849*, 2025.

[21] Chengpeng Li, Zhengyang Tang, Ziniu Li, Mingfeng Xue, Keqin Bao, Tian Ding, Ruoyu Sun, Benyou Wang, Xiang Wang, Junyang Lin, and Dayiheng Liu. Teaching language models to reason with tools. In *The Thirty-ninth Annual Conference on Neural Information Processing Systems*, 2025. URL https://openreview.net/forum?id=kRZVz1qEqa.

[22] Niklas Muennighoff, Zitong Yang, Weijia Shi, Xiang Lisa Li, Li Fei-Fei, Hannaneh Hajishirzi, Luke Zettlemoyer, Percy Liang, Emmanuel Candès, and Tatsunori Hashimoto. s1: Simple test-time scaling. *arXiv preprint arXiv:2501.19393*, 2025.

[23] Yixin Ye, Zhen Huang, Yang Xiao, Ethan Chern, Shijie Xia, and Pengfei Liu. Limo: Less is more for reasoning. *arXiv preprint arXiv:2502.03387*, 2025.

[24] Dacheng Li, Shiyi Cao, Tyler Griggs, Shu Liu, Xiangxi Mo, Eric Tang, Sumanth Hegde, Kourosh Hakhamaneshi, Shishir G Patil, Matei Zaharia, et al. Llms can easily learn to reason from demonstrations structure, not content, is what matters! *arXiv preprint arXiv:2502.07374*, 2025.

[25] Liang Wen, Yunke Cai, Fenrui Xiao, Xin He, Qi An, Zhenyu Duan, Yimin Du, Junchen Liu, Lifu Tang, Xiaowei Lv, et al. Light-R1: Curriculum sft, dpo and rl for long cot from scratch and beyond. *arXiv preprint arXiv:2503.10460*, 2025.

[26] Etash Guha, Ryan Marten, Sedrick Keh, Negin Raoof, Georgios Smyrnis, Hritik Bansal, Marianna Nezhurina, Jean Mercat, Trung Vu, Zayne Sprague, et al. Openthoughts: Data recipes for reasoning models. *arXiv preprint arXiv:2506.04178*, 2025.

[27] Guangming Sheng, Chi Zhang, Zilingfeng Ye, Xibin Wu, Wang Zhang, Ru Zhang, Yanghua Peng, Haibin Lin, and Chuan Wu. Hybridflow: A flexible and efficient rlhf framework. In *Proceedings of the Twentieth European Conference on Computer Systems*, pages 1279–1297, 2025.

[28] Dan Hendrycks, Collin Burns, Saurav Kadavath, Akul Arora, Steven Basart, Eric Tang, Dawn Song, and Jacob Steinhardt. Measuring mathematical problem solving with the math dataset. *arXiv preprint arXiv:2103.03874*, 2021.

[29] Jia Li, Edward Beeching, Lewis Tunstall, Ben Lipkin, Roman Soletskyi, Shengyi Costa Huang, Kashif Rasul, Longhui Yu, Albert Jiang, Ziju Shen, Zihan Qin, Bin Dong, Li Zhou, Yann Fleureau, Guillaume Lample, and Stanislas Polu. Numinamath, 2024. URL https://github.com/project-numina/aimo-progress-prize.

[30] Aitor Lewkowycz, Anders Andreassen, David Dohan, Ethan Dyer, Henryk Michalewski, Vinay Ramasesh, Ambrose Slone, Cem Anil, Imanol Schlag, Theo Gutman-Solo, et al. Solving quantitative reasoning problems with language models. *Advances in neural information processing systems*, 35:3843–3857, 2022.

[31] Chaoqun He, Renjie Luo, Yuzhuo Bai, Shengding Hu, Zhen Leng Thai, Junhao Shen, Jinyi Hu, Xu Han, Yujie Huang, Yuxiang Zhang, Jie Liu, Lei Qi, Zhiyuan Liu, and Maosong Sun. Olympiadbench: A challenging benchmark for promoting agi with olympiad-level bilingual multimodal scientific problems. *arXiv preprint arXiv:2402.14008*, 2024.

[32] Shivam Agarwal, Zimin Zhang, Lifan Yuan, Jiawei Han, and Hao Peng. The unreasonable effectiveness of entropy minimization in llm reasoning. *arXiv preprint arXiv:2505.15134*, 2025.

[33] Leo Gao, Jonathan Tow, Baber Abbasi, Stella Biderman, Sid Black, Anthony DiPofi, Charles Foster, Laurence Golding, Jeffrey Hsu, Alain Le Noac'h, Haonan Li, Kyle McDonell, Niklas Muennighoff, Chris Ociepa, Jason Phang, Laria Reynolds, Hailey Schoelkopf, Aviya Skowron, Lintang Sutawika, Eric Tang, Anish Thite, Ben Wang, Kevin Wang, and Andy Zou. The language model evaluation harness, 07 2024. URL https://zenodo.org/records/12608602.

[34] Jeffrey Zhou, Tianjian Lu, Swaroop Mishra, Siddhartha Brahma, Sujoy Basu, Yi Luan, Denny Zhou, and Le Hou. Instruction-following evaluation for large language models. *arXiv preprint arXiv:2311.07911*, 2023.

[35] Yubo Wang, Xueguang Ma, Ge Zhang, Yuansheng Ni, Abhranil Chandra, Shiguang Guo, Weiming Ren, Aaran Arulraj, Xuan He, Ziyan Jiang, et al. Mmlu-pro: a more robust and challenging multi-task language understanding benchmark. In *Proceedings of the 38th International Conference on Neural Information Processing Systems*, pages 95266–95290, 2024.

[36] Mirac Suzgun, Nathan Scales, Nathanael Schärli, Sebastian Gehrmann, Yi Tay, Hyung Won Chung, Aakanksha Chowdhery, Quoc V Le, Ed H Chi, Denny Zhou, , and Jason Wei. Challenging big-bench tasks and whether chain-of-thought can solve them. *arXiv preprint arXiv:2210.09261*, 2022.

[37] David Rein, Betty Li Hou, Asa Cooper Stickland, Jackson Petty, Richard Yuanzhe Pang, Julien Dirani, Julian Michael, and Samuel R Bowman. Gpqa: A graduate-level google-proof q&a benchmark. In *First Conference on Language Modeling*, 2024.

[38] Zayne Sprague, Xi Ye, Kaj Bostrom, Swarat Chaudhuri, and Greg Durrett. Musr: Testing the limits of chain-of-thought with multistep soft reasoning. *ICLR*, 2024.

[39] Yang Yue, Zhiqi Chen, Rui Lu, Andrew Zhao, Zhaokai Wang, Shiji Song, and Gao Huang. Does reinforcement learning really incentivize reasoning capacity in llms beyond the base model? *arXiv preprint arXiv:2504.13837*, 2025.

[40] Shenzhi Wang, Le Yu, Chang Gao, Chujie Zheng, Shixuan Liu, Rui Lu, Kai Dang, Xionghui Chen, Jianxin Yang, Zhenru Zhang, et al. Beyond the 80/20 rule: High-entropy minority tokens drive effective reinforcement learning for llm reasoning. *arXiv preprint arXiv:2506.01939*, 2025.

[41] Yanda Chen, Joe Benton, Ansh Radhakrishnan, Jonathan Uesato, Carson Denison, John Schulman, Arushi Somani, Peter Hase, Misha Wagner, Fabien Roger, et al. Reasoning models don't always say what they think. *arXiv preprint arXiv:2505.05410*, 2025.

[42] Parshin Shojaee, Iman Mirzadeh, Keivan Alizadeh, Maxwell Horton, Samy Bengio, and Mehrdad Farajtabar. The illusion of thinking: Understanding the strengths and limitations of reasoning models via the lens of problem complexity. *arXiv preprint arXiv:2506.06941*, 2025.

[43] Rulin Shao, Shuyue Stella Li, Rui Xin, Scott Geng, Yiping Wang, Sewoong Oh, Simon Shaolei Du, Nathan Lambert, Sewon Min, Ranjay Krishna, Yulia Tsvetkov, Hannaneh Hajishirzi, Pang Wei Koh, and Luke Zettlemoyer. Spurious rewards: Rethinking training signals in rlvr. *arXiv preprint arXiv:2506.10947*, 2025.

[44] Ganqu Cui, Yuchen Zhang, Jiacheng Chen, Lifan Yuan, Zhi Wang, Yuxin Zuo, Haozhan Li, Yuchen Fan, Huayu Chen, Weize Chen, Zhiyuan Liu, Hao Peng, Lei Bai, Wanli Ouyang, Yu Cheng, Bowen Zhou, and Ning Ding. The entropy mechanism of reinforcement learning for reasoning language models. *arXiv preprint arXiv:2505.22617*, 2025.

[45] Audrey Huang, Adam Block, Dylan J Foster, Dhruv Rohatgi, Cyril Zhang, Max Simchowitz, Jordan T Ash, and Akshay Krishnamurthy. Self-improvement in language models: The sharpening mechanism. *arXiv preprint arXiv:2412.01951*, 2024.

[46] Bingxiang He, Yuxin Zuo, Zeyuan Liu, Shangziqi Zhao, Zixuan Fu, Junlin Yang, Cheng Qian, Kaiyan Zhang, Yuchen Fan, Ganqu Cui, et al. How far can unsupervised rlvr scale llm training? *arXiv preprint arXiv:2603.08660*, 2026.

[47] Aayush Karan and Yilun Du. Reasoning with sampling: Your base model is smarter than you think. *arXiv preprint arXiv:2510.14901*, 2025.

[48] Yang Sui, Yu-Neng Chuang, Guanchu Wang, Jiamu Zhang, Tianyi Zhang, Jiayi Yuan, Hongyi Liu, Andrew Wen, Shaochen Zhong, Na Zou, et al. Stop overthinking: A survey on efficient reasoning for large language models. *arXiv preprint arXiv:2503.16419*, 2025.

[49] Eric Zelikman, Yuhuai Wu, Jesse Mu, and Noah Goodman. Star: Bootstrapping reasoning with reasoning. *Advances in Neural Information Processing Systems*, 35:15476–15488, 2022.

[50] Yizhong Wang, Yeganeh Kordi, Swaroop Mishra, Alisa Liu, Noah A Smith, Daniel Khashabi, and Hannaneh Hajishirzi. Self-instruct: Aligning language models with self-generated instructions. *arXiv preprint arXiv:2212.10560*, 2022.

[51] Tu Vu, Minh-Thang Luong, Quoc Le, Grady Simon, and Mohit Iyyer. Strata: Self-training with task augmentation for better few-shot learning. In *Proceedings of the 2021 Conference on Empirical Methods in Natural Language Processing*, pages 5715–5731, 2021.

[52] Yuxin Zuo, Kaiyan Zhang, Li Sheng, Shang Qu, Ganqu Cui, Xuekai Zhu, Haozhan Li, Yuchen Zhang, Xinwei Long, Ermo Hua, et al. Ttrl: Test-time reinforcement learning. *arXiv preprint arXiv:2504.16084*, 2025.

[53] Siyan Zhao, Zhihui Xie, Mengchen Liu, Jing Huang, Guan Pang, Feiyu Chen, and Aditya Grover. Self-distilled reasoner: On-policy self-distillation for large language models. *arXiv preprint arXiv:2601.18734*, 2026.

[54] Idan Shenfeld, Mehul Damani, Jonas Hübotter, and Pulkit Agrawal. Self-distillation enables continual learning. *arXiv preprint arXiv:2601.19897*, 2026.

[55] Jonas Hübotter, Frederike Lübeck, Lejs Behric, Anton Baumann, Marco Bagatella, Daniel Marta, Ido Hakimi, Idan Shenfeld, Thomas Kleine Buening, Carlos Guestrin, et al. Reinforcement learning via self-distillation. *arXiv preprint arXiv:2601.20802*, 2026.

[56] Jared Kaplan, Sam McCandlish, Tom Henighan, Tom B Brown, Benjamin Chess, Rewon Child, Scott Gray, Alec Radford, Jeffrey Wu, and Dario Amodei. Scaling laws for neural language models. *arXiv preprint arXiv:2001.08361*, 2020.

[57] Jordan Hoffmann, Sebastian Borgeaud, Arthur Mensch, Elena Buchatskaya, Trevor Cai, Eliza Rutherford, Diego de Las Casas, Lisa Anne Hendricks, Johannes Welbl, Aidan Clark, et al. Training compute-optimal large language models. In *Proceedings of the 36th International Conference on Neural Information Processing Systems*, pages 30016–30030, 2022.

Contents

A Additional Related Works

Due to the rapid growth of research on LLM reasoning, we provide a non-exhaustive overview.

There is a broad line of work analyzing what reasoning post-training is actually doing. The work [39] asks whether RL essentially creates new reasoning ability beyond the base model or mainly exposes capabilities already present. [40] highlights the disproportionate importance of a minority of high-entropy tokens in effective reasoning RL. [41] studies the gap between a model's internal reasoning process and the reasoning it explicitly verbalizes. [42] examines apparent reasoning gains through the lens of problem complexity and cautions against over-interpreting them. [43] revisits whether commonly used reward signals are always aligned with the intended reasoning objective. [32, 44] study entropy reduction as an important mechanism in reasoning post-training. [45] studies sharpening as a self-improvement mechanism, where post-training uses the model's own verification signal to shift probability mass toward high-quality sequences covered by the base model. [46] analyzes unsupervised RLVR, showing that intrinsic rewards mainly sharpen the model's initial distribution and can collapse when confidence is misaligned with correctness. Other directions include obtaining gains from extremely limited supervision [19], exploiting sampling itself as a source of reasoning improvement [47], and surveying methods for making reasoning more efficient [48].

Additionally, STaR [49] bootstraps reasoning by generating rationales and then retaining trajectories using gold-answer correctness, while Self-Instruct [50] expands instruction-following data through self-generated instructions and responses starting from human-written seeds. STraTA [51] combines task augmentation with pseudo-label-based self-training, showing that unlabeled target-task text can improve few-shot sample efficiency. TTRL [52] uses majority-vote signals from repeated test-time sampling as rewards for reinforcement learning on unlabeled reasoning data. Recent self-distillation methods further study online or on-policy variants. OPSD [53] studies on-policy self-distillation for large language models, SDFT [54] explores self-distillation as a mechanism for continual learning, and SDPO [55] combines reinforcement learning with self-distillation. EM-FT [32] is a reward-free direction, which trains on unlabeled model outputs by directly minimizing token-level entropy. Relative to these works, SePT focuses on reward-free online post-training for reasoning using only self-generated responses and the simple and standard SFT training objective, without rewards, verifiers, or teacher-provided solution traces.

B Detailed Experiment Setup

B.1 Training Configuration

General Parameters. All models are trained on a cluster of 8 NVIDIA A800 GPUs. Training uses the AdamW optimizer with weight decay 0.01. A constant learning rate of 1e-7 is used for SePT and 5e-7 for RLVR. Training runs for 3 epochs in all cases with a 10-step warmup. Following recent works [11, 12, 18], the KL divergence regularizer is disabled to ensure a direct comparison of the core learning algorithms.

Batching and Gradient Updates. All experiments share a common batching structure. The global batch size is set to 128 prompts per training step. These prompts are processed over two gradient updates, with a mini-batch size of 64 prompts per update. The micro-batch size can be up to 32 sequences per GPU for each forward/backward pass. The total number of sequences per update and the resulting number of gradient accumulation steps depend on the number of rollouts (G), which is method-specific.

Method-Specific Configurations.

SePT. Our method is configured for efficiency, using a single rollout per prompt ($G = 1$). Consequently, each gradient update processes 64 sequences, so no gradient accumulation is needed. SePT uses low-τ_s temperature regime in the main training stage. The temperature τ_s is annealed according to

$$\tau_s(u) = \begin{cases} \tau_{s,\text{start}} + \dfrac{u}{2}\left(1 - \tau_{s,\text{start}}\right), & 0 \le u < 2, \\ 1, & 2 \le u \le 3, \end{cases}$$

where u denotes training progress measured in epochs. Thus, over 3 training epochs, τ_s is linearly annealed from $\tau_{s,\text{start}}$ to 1 during the first 2 epochs, and we use the neutral setting $\tau_s = \tau_t = 1$ for the final epoch. In particular, we set $\tau_{s,\text{start}} = 0.6$ for specialized math models (Qwen-2.5-Math-7B and DeepSeek-Math-7B-Instruct) and $\tau_{s,\text{start}} = 0.9$ for other general-purpose models.

GRPO. The GRPO baseline uses the standard 8 rollouts per prompt ($G = 8$). Each gradient update processes a total of $64 \times 8 = 512$ sequences. Given the micro-batch size of 32 and 8 GPUs, this requires 2 gradient accumulation steps per update ($512/(32 \times 8) = 2$).

EM-FT. We adopt the default hyperparameters in the verl framework for EM-FT (the same as SePT) and implement this method by replacing the cross-entropy objective in SePT with token-level entropy minimization on the sampled trajectories. We use the same learning rate as that of SePT for EM-FT.

B.2 Evaluation Configuration

Pass@k Metric. The direct estimation of pass@k by generating only k samples per problem can result in high variance. To achieve a more stable and reliable metric, we utilize the unbiased estimation method detailed in [39].

This approach involves generating a larger number of samples, n (where $n \ge k$), for each problem q_i in the evaluation dataset $\mathcal{D}$. The number of correct samples among these is then counted and denoted as c_i.

The unbiased estimator for pass@k across the dataset is subsequently calculated using the following formula:

$$\text{pass@k} := \mathbb{E}_{q_i \sim \mathcal{D}}\left[1 - \frac{\binom{n-c_i}{k}}{\binom{n}{k}}\right] \tag{2}$$

This method provides a significant advantage, as it allows for the low-variance estimation of pass@k for all values of $k \le n$ from a single set of generated samples.

B.3 Verifier

The selection of a verifier is an important factor for the final score. Various verifiers may use different methods for parsing answers (parser) and assessing correctness (grader). Considering the trade-off between response time and accuracy, we evaluated multiple verifiers and chose the math verifier presented in [44].

B.4 Chat Template

Here, "{reasoning prompt}" is always “Please reason step by step, and put your final answer within \boxed{}.”:

Qwen-Base and Math template
<|im_start|>system\n{reasoning prompt}<|im_end|>\n<|im_start|>user\n
{question}<|im_end|>\n<|im_start|>assistant\n
Qwen-Base-Instruct template
<|im_start|>system\n\nYou are Qwen, created by Alibaba Cloud. You are
a helpful assistant.<|im_end|>\n<|im_start|>user\n{question} {reasoning
prompt}<|im_end|>\n<|im_start|>assistant\n
DeepSeek-Math-Instruct template
<|begin_of_sentence|>\n\n{reasoning prompt}\n\nUser: {question}\n\nAssistant:
Llama-3.1-8B-Instruct template
<|begin_of_text|><|start_header_id|>system<|end_header_id|>\n\n
{reasoning prompt}<|eot_id|><|start_header_id|>user<|end_header_id|>\n\n
{question}<|eot_id|><|start_header_id|>assistant<|end_header_id|>

Figure 4 Chat templates for different models.

B.5 FLOPs Accounting

We report average FLOPs per update for the DSR runs. Following the standard dense-transformer compute estimate used in scaling-law analyses [56, 57], an inference forward pass over D tokens costs approximately $2PD$ FLOPs, where P is the number of model parameters. A training forward-backward pass costs approximately $6PD$ FLOPs. We use $P = 7 \times 10^9$ for all 7B dense models.

Let $\bar{D}$ denote the average number of prompt-response tokens processed per update. For SePT, one update consists of self-generation followed by supervised finetuning on the generated responses. Thus,

$$C_{\text{SePT}} = 2P\bar{D}_{\text{SePT}} + 6P\bar{D}_{\text{SePT}} = 8P\bar{D}_{\text{SePT}}. \tag{3}$$

For GRPO, one update consists of rollout generation, old-policy log-probability computation, and the actor update. In principle, the old-policy log probabilities could be cached during rollout generation; however, verl recomputes them separately, so we count this as an additional forward pass. Since the KL regularizer is disabled, no reference-model forward pass is included. Thus,

$$C_{\text{GRPO}} = 2P\bar{D}_{\text{GRPO}} + 2P\bar{D}_{\text{GRPO}} + 6P\bar{D}_{\text{GRPO}} = 10P\bar{D}_{\text{GRPO}}. \tag{4}$$

C Detailed Figures and Tables

This section provides detailed scores of each benchmark for each experiment.

C.1 Decoding Temperature Sweeps on Other Models

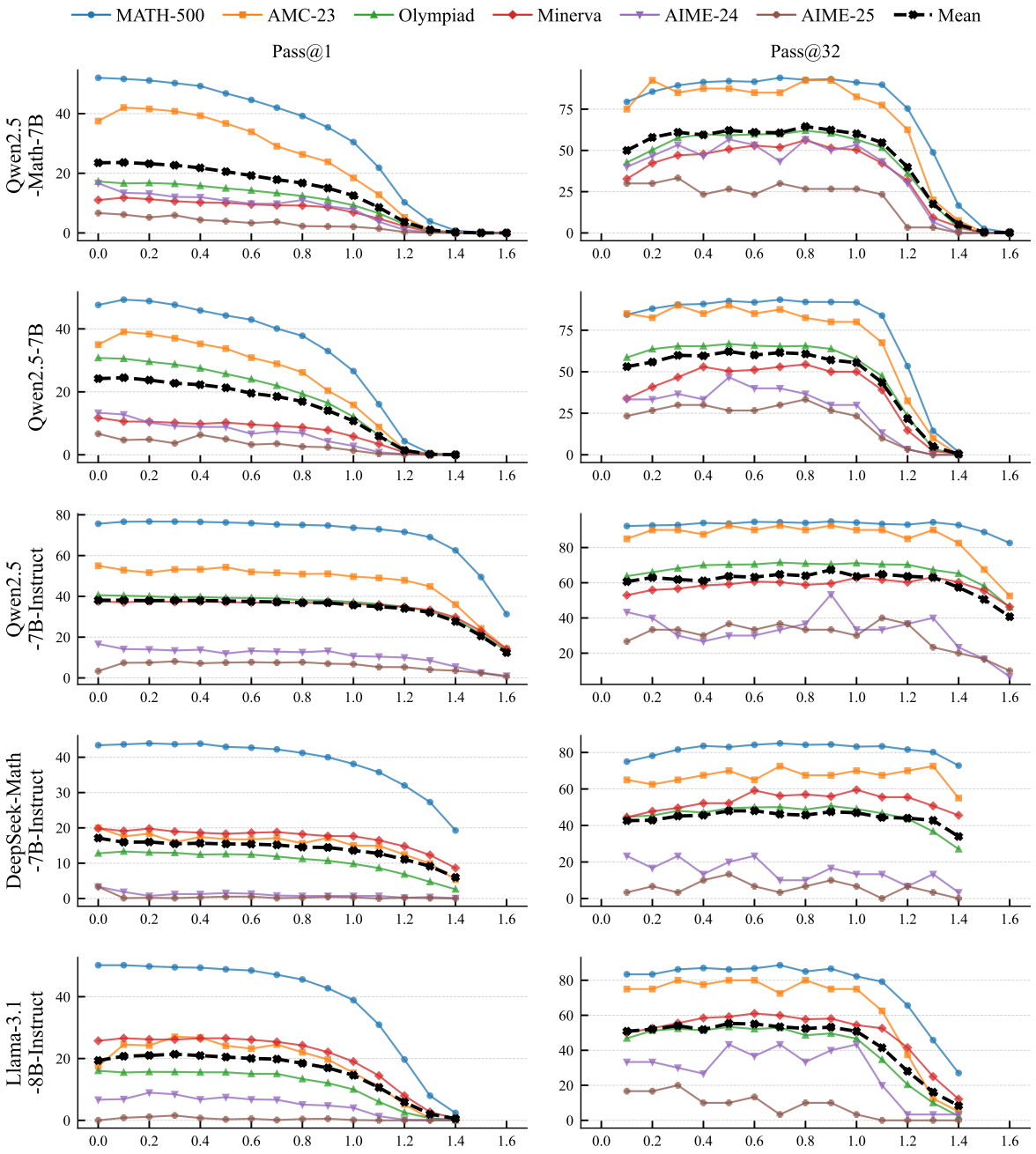


Figure 5 Pass@1 and Pass@32 across different decoding temperatures on different models.

C.2 SePT Improves Over the No-Training Baseline

Table 8 Full benchmark-level Pass@k results in Table 1.

Model	Method	Benchmark	$k=1$	$k=8$	$k=32$
Qwen2.5-Math-7B	Base Model	MATH-500	30.4	78.2	91.2
		AMC-23	18.4	61.7	82.5
		Minerva	6.9	31.0	50.4
		Olympiad	9.3	36.6	56.6
		AIME-24	7.9	32.2	53.3
		AIME-25	2.1	12.6	26.7
		Avg.	12.5	42.0	60.1
	Baseline	MATH-500	50.2	79.2	89.4
		AMC-23	40.8	74.4	85.0
		Minerva	10.6	32.6	47.1
		Olympiad	16.5	40.9	57.7
		AIME-24	12.1	35.6	53.3
		AIME-25	5.9	21.4	33.3
		Avg.	22.7	47.3	61.0
	SePT	MATH-500	77.2	90.1	93.0
		AMC-23	59.0	81.4	90.0
		Minerva	35.5	53.6	61.8
		Olympiad	39.6	60.6	69.1
		AIME-24	14.0	33.2	53.3
		AIME-25	11.7	27.5	40.0
		Avg.	39.5	57.7	67.9
Qwen2.5-7B	Base Model	MATH-500	26.6	75.9	91.8
		AMC-23	15.9	58.6	80.0
		Minerva	5.8	27.6	50.0
		Olympiad	12.1	41.1	57.5
		AIME-24	2.8	13.9	30.0
		AIME-25	1.4	8.0	23.3
		Avg.	10.7	37.5	55.4
	Baseline	MATH-500	44.2	83.5	92.6
		AMC-23	33.8	75.1	90.0
		Minerva	10.2	34.8	50.4
		Olympiad	25.7	53.6	66.7
		AIME-24	8.9	26.9	46.7
		AIME-25	5.0	18.5	26.7
		Avg.	21.3	48.7	62.2
	SePT	MATH-500	70.5	89.1	93.6
		AMC-23	45.6	78.2	90.0
		Minerva	25.5	50.3	60.3
		Olympiad	34.9	57.7	68.2
		AIME-24	9.8	24.0	36.7
		AIME-25	7.3	28.4	43.3
		Avg.	32.3	54.6	65.3
Qwen2.5-7B -Instruct	Base Model	MATH-500	73.6	90.1	94.2
		AMC-23	49.7	79.8	90.0
		Minerva	36.2	54.4	62.9
		Olympiad	37.2	60.8	71.2
		AIME-24	10.7	24.7	33.3
		AIME-25	6.8	23.4	30.0
		Avg.	35.7	55.5	63.6
	Baseline	MATH-500	74.7	90.4	94.8

Model	Method	Benchmark	$k=1$	$k=8$	$k=32$
		AMC-23	51.1	80.6	92.5
		Minerva	37.1	53.6	59.6
		Olympiad	38.0	60.4	70.4
		AIME-24	13.2	30.8	53.3
		AIME-25	7.0	23.7	33.3
		Avg.	36.8	56.6	67.3
	SePT	MATH-500	74.9	90.6	95.6
		AMC-23	49.8	81.5	97.5
		Minerva	36.8	54.2	61.4
		Olympiad	38.0	59.8	69.5
		AIME-24	11.6	26.6	43.3
		AIME-25	8.2	27.9	46.7
		Avg.	36.6	56.8	69.0
DeepSeek-Math -7B-Instruct	Base Model	MATH-500	38.1	69.7	83.2
		AMC-23	15.0	44.7	70.0
		Minerva	17.6	44.5	59.6
		Olympiad	9.8	31.4	49.0
		AIME-24	0.7	5.1	13.3
		AIME-25	0.3	2.3	6.7
		Avg.	13.6	33.0	47.0
	Baseline	MATH-500	42.7	71.5	84.2
		AMC-23	16.7	44.7	65.0
		Minerva	18.6	44.0	59.2
		Olympiad	12.4	35.0	49.9
		AIME-24	1.4	9.3	23.3
		AIME-25	0.5	3.5	6.7
		Avg.	15.4	34.6	48.1
	SePT	MATH-500	42.4	71.2	83.6
		AMC-23	18.5	48.7	72.5
		Minerva	19.7	44.4	56.2
		Olympiad	12.7	34.0	49.5
		AIME-24	1.4	9.3	23.3
		AIME-25	0.7	5.5	16.7
		Avg.	15.9	35.5	50.3

C.3 Comparison with Other Training-based Methods

Table 9 Full benchmark-level Pass@k results in Table 2.

Model	Benchmark	Method	Data	$k=1$	$k=8$	$k=32$
Qwen2.5-Math-7B	MATH-500	Baseline	—	50.2	79.2	89.4
		GRPO	OTM	76.3	90.4	94.2
			DSR	80.9	90.4	93.8
		SePT	OTM	78.1	90.7	93.8
			DSR	77.2	90.1	93.0
	AMC-23	Baseline	—	40.8	74.4	85.0
		GRPO	OTM	58.8	79.8	90.0
			DSR	63.8	81.6	92.5
		SePT	OTM	62.0	83.9	92.5
			DSR	59.0	81.4	90.0
	Minerva	Baseline	—	10.6	32.6	47.1
		GRPO	OTM	34.0	51.7	61.0
			DSR	37.3	50.9	57.7
		SePT	OTM	35.8	52.9	58.8
			DSR	35.5	53.6	61.8
	Olympiad	Baseline	—	16.5	40.9	57.7
		GRPO	OTM	40.1	60.2	69.7
			DSR	41.6	60.6	68.8
		SePT	OTM	40.6	60.3	68.5
			DSR	39.6	60.6	69.1
	AIME-24	Baseline	—	12.1	35.6	53.3
		GRPO	OTM	15.3	42.8	66.7
			DSR	24.6	53.4	66.7
		SePT	OTM	14.1	33.8	46.7
			DSR	14.0	33.2	53.3
	AIME-25	Baseline	—	5.9	21.4	33.3
		GRPO	OTM	12.8	29.3	46.7
			DSR	14.6	34.2	50.0
		SePT	OTM	14.5	30.1	36.7
			DSR	11.7	27.5	40.0
	Avg.	Baseline	—	22.7	47.3	61.0
		GRPO	OTM	39.5	59.0	71.4
			DSR	43.8	61.8	71.6
		SePT	OTM	40.8	58.6	66.2
			DSR	39.5	57.7	67.9
DeepSeek-Math -7B-Instruct	MATH-500	Baseline	—	42.7	71.5	84.2
		GRPO	OTM	44.2	71.5	83.0
			DSR	47.6	73.6	84.4
		SePT	OTM	42.6	71.3	84.2
			DSR	42.4	71.2	83.6
	AMC-23	Baseline	—	16.7	44.7	65.0
		GRPO	OTM	19.9	49.0	75.0

Model	Benchmark	Method	Data	$k=1$	$k=8$	$k=32$
			DSR	21.2	55.0	77.5
		SePT	OTM	17.9	47.9	65.0
			DSR	18.5	48.7	72.5
	Minerva	Baseline	—	18.6	44.0	59.2
		GRPO	OTM	21.7	46.1	60.3
			DSR	22.3	46.6	59.2
		SePT	OTM	18.9	42.9	55.1
			DSR	19.7	44.4	56.2
	Olympiad	Baseline	—	12.4	35.0	49.9
		GRPO	OTM	13.9	34.3	47.7
			DSR	15.6	37.4	51.6
		SePT	OTM	12.9	34.0	48.4
			DSR	12.7	34.0	49.5
	AIME-24	Baseline	—	1.4	9.3	23.3
		GRPO	OTM	1.7	8.3	16.7
			DSR	2.1	10.1	20.0
		SePT	OTM	1.7	9.5	16.7
			DSR	1.4	9.3	23.3
	AIME-25	Baseline	—	0.5	3.5	6.7
		GRPO	OTM	0.2	1.7	6.7
			DSR	0.4	3.3	13.3
		SePT	OTM	0.6	5.0	20.0
			DSR	0.7	5.5	16.7
	Avg.	Baseline	—	15.4	34.6	48.1
		GRPO	OTM	16.9	35.1	48.2
			DSR	18.2	37.7	51.0
		SePT	OTM	15.8	35.1	48.2
			DSR	15.9	35.5	50.3

Table 10 Full benchmark-level Pass@k results in Figure 3.

Model	Method	Benchmark	$k=1$	$k=8$	$k=32$
Qwen2.5-Math-7B	Baseline	MATH-500	50.2	79.2	89.4
		AMC-23	40.8	74.4	85.0
		Minerva	10.6	32.6	47.1
		Olympiad	16.5	40.9	57.7
		AIME-24	12.1	35.6	53.3
		AIME-25	5.9	21.4	33.3
		Avg.	22.7	47.3	61.0
	SePT	MATH-500	77.2	90.1	93.0
		AMC-23	59.0	81.4	90.0
		Minerva	35.5	53.6	61.8
		Olympiad	39.6	60.6	69.1
		AIME-24	14.0	33.2	53.3
		AIME-25	11.7	27.5	40.0
		Avg.	39.5	57.7	67.9
	EM-FT	MATH-500	61.1	89.4	94.4
		AMC-23	47.6	80.2	87.5
		Minerva	18.7	47.9	60.7

Model	**Method**	**Benchmark**	$k=1$	$k=8$	$k=32$
		Olympiad	29.4	57.8	71.0
		AIME-24	15.6	44.2	53.3
		AIME-25	6.8	21.5	36.7
		Avg.	29.9	56.8	67.3
DeepSeek-Math -7B-Instruct	Baseline	MATH-500	42.7	71.5	84.2
		AMC-23	16.7	44.7	65.0
		Minerva	18.6	44.0	59.2
		Olympiad	12.4	35.0	49.9
		AIME-24	1.4	9.3	23.3
		AIME-25	0.5	3.5	6.7
		Avg.	15.4	34.6	48.1
	SePT	MATH-500	42.4	71.2	83.6
		AMC-23	18.5	48.7	72.5
		Minerva	19.7	44.4	56.2
		Olympiad	12.7	34.0	49.5
		AIME-24	1.4	9.3	23.3
		AIME-25	0.7	5.5	16.7
		Avg.	15.9	35.5	50.3
	EM-FT	MATH-500	43.1	70.7	83.6
		AMC-23	18.5	51.1	72.5
		Minerva	19.4	43.7	55.1
		Olympiad	12.9	35.1	50.1
		AIME-24	1.0	6.8	16.7
		AIME-25	0.1	0.8	3.3
		Avg.	15.9	34.7	46.9

C.4 Effect of Online Data Refresh: Comparison with SePT (Offline)

Table 11 Full benchmark-level Pass@k results in Table 5.

Benchmark	Method	$k=1$	$k=8$	$k=32$
MATH-500	Baseline	50.2	79.2	89.4
	SePT	77.2	90.1	93.0
	SePT (Offline)	47.6	85.7	94.2
AMC-23	Baseline	40.8	74.4	85.0
	SePT	59.0	81.4	90.0
	SePT (Offline)	36.4	75.4	90.0
Minerva	Baseline	10.6	32.6	47.1
	SePT	35.5	53.6	61.8
	SePT (Offline)	10.9	38.7	56.6
Olympiad	Baseline	16.5	40.9	57.7
	SePT	39.6	60.6	69.1
	SePT (Offline)	15.7	45.3	63.4
AIME-24	Baseline	12.1	35.6	53.3
	SePT	14.0	33.2	53.3
	SePT (Offline)	11.6	38.1	53.3
AIME-25	Baseline	5.9	21.4	33.3
	SePT	11.7	27.5	40.0
	SePT (Offline)	3.4	18.3	33.3
Avg.	Baseline	22.7	47.3	61.0
	SePT	39.5	57.7	67.9
	SePT (Offline)	21.0	50.2	65.2

C.5 Effect of Temperature Dynamics

Table 12 Full benchmark-level Pass@k results in Table 6.

Benchmark	Method	$k=1$	$k=8$	$k=32$
MATH-500	Baseline	50.2	79.2	89.4
	$\tau_s = \tau_t$	45.4	86.5	94.0
	$\tau_s < \tau_t$	77.2	90.1	93.0
AMC-23	Baseline	40.8	74.4	85.0
	$\tau_s = \tau_t$	31.1	76.5	90.0
	$\tau_s < \tau_t$	59.0	81.4	90.0
Minerva	Baseline	10.6	32.6	47.1
	$\tau_s = \tau_t$	9.8	37.6	56.6
	$\tau_s < \tau_t$	35.5	53.6	61.8
Olympiad	Baseline	16.5	40.9	57.7
	$\tau_s = \tau_t$	15.1	47.1	65.1
	$\tau_s < \tau_t$	39.6	60.6	69.1
AIME-24	Baseline	12.1	35.6	53.3
	$\tau_s = \tau_t$	11.1	38.3	56.7
	$\tau_s < \tau_t$	14.0	33.2	53.3
AIME-25	Baseline	5.9	21.4	33.3
	$\tau_s = \tau_t$	3.3	14.6	23.3
	$\tau_s < \tau_t$	11.7	27.5	40.0
Avg.	Baseline	22.7	47.3	61.0
	$\tau_s = \tau_t$	19.3	50.1	64.3

Benchmark	**Method**	$k = 1$	$k = 8$	$k = 32$
	$\tau_s < \tau_t$	39.5	57.7	67.9

C.6 Limitation

Table 13 Full benchmark-level Pass@k results in Table 7.

Model	Benchmark	Method	Data	$k=1$	$k=8$	$k=32$
Llama-3.1 -8B-Instruct	MATH-500	Base Model	—	38.9	70.1	82.2
		Baseline	—	48.9	76.2	86.2
		SePT	DSR	49.2	75.8	85.8
			OTM	47.9	74.0	83.4
		GRPO	DSR	55.6	78.1	87.0
			OTM	47.7	76.2	86.0
	AMC-23	Base Model	—	15.4	50.6	75.0
		Baseline	—	24.1	56.6	80.0
		SePT	DSR	22.9	56.6	82.5
			OTM	25.7	60.0	82.5
		GRPO	DSR	33.4	61.7	75.0
			OTM	24.2	58.4	77.5
	Minerva	Base Model	—	19.0	43.2	54.4
		Baseline	—	26.6	47.5	59.2
		SePT	DSR	25.6	45.4	54.4
			OTM	24.3	44.6	55.5
		GRPO	DSR	29.4	50.0	59.6
			OTM	25.4	48.5	59.6
	Olympiad	Base Model	—	10.0	30.2	46.7
		Baseline	—	15.6	38.4	53.3
		SePT	DSR	16.2	38.9	54.4
			OTM	15.7	36.5	49.8
		GRPO	DSR	21.4	40.1	51.4
			OTM	15.6	35.9	49.0
	AIME-24	Base Model	—	4.1	22.1	43.3
		Baseline	—	7.5	24.4	43.3
		SePT	DSR	5.7	18.2	30.0
			OTM	6.0	21.5	36.7
		GRPO	DSR	10.9	29.9	43.3
			OTM	5.2	19.4	30.0
	AIME-25	Base Model	—	0.1	0.8	3.3
		Baseline	—	0.3	2.5	10.0
		SePT	DSR	0.5	4.0	13.3
			OTM	0.8	5.7	16.7
		GRPO	DSR	0.7	4.3	10.0
			OTM	0.1	0.8	3.3
	Avg.	Base Model	—	14.6	36.2	50.8
		Baseline	—	20.5	40.9	55.3
		SePT	DSR	20.0	39.8	53.4
			OTM	20.1	40.4	54.1
		GRPO	DSR	25.2	44.0	54.4
			OTM	19.7	39.9	50.9

Table 14 Contamination checks used in the contamination audit. The exact-normalized check is added by us. The other checks follow Light-R1 and OpenThoughts.

Check	Source	Description
Exact-normalized	Ours	Exact match after text normalization
Digit-insensitive exact	Light-R1	Exact match after removing digits
32-word n-gram	Light-R1	Shared contiguous 32-word span
Indel 75	OpenThoughts	Normalized Indel similarity ≥ 75
Qwen-token 13-gram	OpenThoughts	Shared 13-token span under Qwen2 tokenizer

Table 15 Final reviewed contamination audit for DSR and OTM against the six evaluation benchmarks. Matched train rows count unique training rows kept after review as confirmed or likely contamination. Matched evaluation questions count unique evaluation questions matched by at least one reviewed-positive training row. The Total row de-duplicates matched training rows across benchmarks.

Training Set	Benchmark	Matched Train Rows	Train Rate	Matched Eval. Questions	Eval. Rate
DSR	MATH-500	145	0.36%	67 / 500	13.40%
	AMC-23	0	0.00%	0 / 40	0.00%
	AIME-24	0	0.00%	0 / 30	0.00%
	AIME-25	0	0.00%	0 / 30	0.00%
	Minerva	0	0.00%	0 / 272	0.00%
	Olympiad	18	0.04%	17 / 675	2.52%
	Total	163	0.40%	84 / 1,547	5.43%
OTM	MATH-500	0	0.00%	0 / 500	0.00%
	AMC-23	0	0.00%	0 / 40	0.00%
	AIME-24	0	0.00%	0 / 30	0.00%
	AIME-25	0	0.00%	0 / 30	0.00%
	Minerva	0	0.00%	0 / 272	0.00%
	Olympiad	11	0.03%	3 / 675	0.44%
	Total	11	0.03%	3 / 1,547	0.19%

D Benchmark Contamination Audit

We audit DSR and OTM against the six evaluation benchmarks to estimate contamination. We extract the question text from each training example and evaluation problem, then apply five lexical checks. As summarized in Table 14, we use one exact-normalized check, two checks following Light-R1 [25], and two checks following OpenThoughts [26].

A pair is first flagged as a candidate if it matches under any check. Then, we apply a review for the flagged candidate. Specifically, exact duplicates and clear long-span overlaps are counted as contamination. Digit-insensitive, Qwen-token, and Indel matches are counted only when the overlap is problem-specific. Matches caused only by answer-format boilerplate, generic definitions, numeric runs, indexed-variable notation, or diagram code are removed as false positives.

Table 15 reports the final reviewed results. DSR has more contamination than OTM, mainly from MATH-500 and Olympiad. In particular, MATH-500 includes exact duplicates and near-duplicate or number-changed problem templates, while OlympiadBench includes clear long-span overlaps. No reviewed-positive DSR contamination remains for AMC-23, AIME-24, AIME-25, or Minerva after removing false positives. OTM has much lower contamination, with reviewed-positive cases only in Olympiad. Thus, DSR has higher contamination than OTM under this audit, and DSR-based gains might be interpreted with this caveat.

E Full Question and Responses from Base and SePT Models

This section provides the full text for the case study visualized in Figure 2. Figure 6 details one incorrect response chosen from all 8 wrong tries from the base model, while Figure 7 shows the corresponding correct response generated after performing SePT.

Example from MATH-500

Problem: The expression $2 \cdot 3 \cdot 4 \cdot 5 + 1$ is equal to 121, since multiplication is carried out before addition. However, we can obtain values other than 121 for this expression if we are allowed to change it by inserting parentheses. For example, we can obtain 144 by writing

$$(2 \cdot (3 \cdot 4)) \cdot (5 + 1) = 144.$$

In total, how many values can be obtained from the expression $2 \cdot 3 \cdot 4 \cdot 5 + 1$ by inserting parentheses? (Note that rearranging terms is not allowed, only inserting parentheses).

Response (Base Model)

To determine how many distinct values can be obtained from the expression $2 \cdot 3 \cdot 4 \cdot 5 + 1$ by inserting parentheses, we need to consider all possible ways of inserting parentheses into the expression.
First, let's evaluate the expression without any parentheses:

$$2 \cdot 3 \cdot 4 \cdot 5 + 1 = 121$$

Now, we need to consider all possible ways of inserting parentheses.

There are five possible ways to insert one set of parentheses: 1. $((2 \cdot 3) \cdot 4) \cdot 5 + 1$ 2. $(2 \cdot (3 \cdot 4)) \cdot 5 + 1$ 3. $(2 \cdot 3) \cdot (4 \cdot 5) + 1$ 4. $2 \cdot ((3 \cdot 4) \cdot 5) + 1$ 5. $2 \cdot (3 \cdot (4 \cdot 5)) + 1$
However, the first and fifth expressions will be the same, as will the second and fourth. This leaves us with three unique ways of inserting parentheses, in addition to the original expression without parentheses.
So we need to evaluate each of these expressions: 1. $((2 \cdot 3) \cdot 4) \cdot 5 + 1 = (6 \cdot 4) \cdot 5 + 1 = 24 \cdot 5 + 1 = 121$ 2. $(2 \cdot (3 \cdot 4)) \cdot 5 + 1 = (2 \cdot 12) \cdot 5 + 1 = 24 \cdot 5 + 1 = 121$ 3. $(2 \cdot 3) \cdot (4 \cdot 5) + 1 = 6 \cdot 20 + 1 = 120 + 1 = 121$ 4. $2 \cdot ((3 \cdot 4) \cdot 5) + 1 = 2 \cdot (12 \cdot 5) + 1 = 2 \cdot 60 + 1 = 120 + 1 = 121$ 5. $2 \cdot (3 \cdot (4 \cdot 5)) + 1 = 2 \cdot (3 \cdot 20) + 1 = 2 \cdot 60 + 1 = 120 + 1 = 121$
From this, we see that all possible ways of inserting parentheses result in the value 121. Therefore, the only distinct value we can obtain is 121.
Hence, the number of distinct values we can obtain from the expression $2 \cdot 3 \cdot 4 \cdot 5 + 1$ by inserting parentheses is:

$$\boxed{1}$$

Figure 6 Full question and the incorrect response generated by the base model, corresponding to the analysis in Figure 2. The soft blue and light gray backgrounds distinguish the initial reasoning prefix (A) from the subsequent flawed suffix (B). This segmentation is because B contains the core reasoning and calculation leading to the answer, while A represents the initial problem setup.

Example from MATH-500

Problem: The expression $2 \cdot 3 \cdot 4 \cdot 5 + 1$ is equal to 121, since multiplication is carried out before addition. However, we can obtain values other than 121 for this expression if we are allowed to change it by inserting parentheses. For example, we can obtain 144 by writing

$$(2 \cdot (3 \cdot 4)) \cdot (5 + 1) = 144.$$

In total, how many values can be obtained from the expression $2 \cdot 3 \cdot 4 \cdot 5 + 1$ by inserting parentheses? (Note that rearranging terms is not allowed, only inserting parentheses).

Response (SePT Model)

To determine how many different values can be obtained from the expression $2 \cdot 3 \cdot 4 \cdot 5 + 1$ by inserting parentheses, we need to consider all possible ways to group the terms. Let's break this down step by step. First, let's list all the possible ways to insert parentheses in the expression $2 \cdot 3 \cdot 4 \cdot 5 + 1$:

1. $(2 \cdot 3 \cdot 4 \cdot 5) + 1$ 2. $2 \cdot (3 \cdot 4 \cdot 5 + 1)$ 3. $2 \cdot 3 \cdot (4 \cdot 5 + 1)$ 4. $2 \cdot (3 \cdot (4 \cdot 5 + 1))$ 5. $(2 \cdot 3 \cdot 4) \cdot (5 + 1)$ 6. $(2 \cdot 3) \cdot (4 \cdot (5 + 1))$ 7. $2 \cdot ((3 \cdot 4) \cdot (5 + 1))$ 8. $((2 \cdot 3) \cdot 4) \cdot (5 + 1)$ 9. $(2 \cdot (3 \cdot 4)) \cdot (5 + 1)$
Now, let's evaluate each of these expressions:
1. $(2 \cdot 3 \cdot 4 \cdot 5) + 1 = 120 + 1 = 121$ 2. $2 \cdot (3 \cdot 4 \cdot 5 + 1) = 2 \cdot (60 + 1) = 2 \cdot 61 = 122$ 3. $2 \cdot 3 \cdot (4 \cdot 5 + 1) = 6 \cdot (20 + 1) = 6 \cdot 21 = 126$ 4. $2 \cdot (3 \cdot (4 \cdot 5 + 1)) = 2 \cdot (3 \cdot 21) = 2 \cdot 63 = 126$ 5. $(2 \cdot 3 \cdot 4) \cdot (5 + 1) = 24 \cdot 6 = 144$ 6. $(2 \cdot 3) \cdot (4 \cdot (5 + 1)) = 6 \cdot (4 \cdot 6) = 6 \cdot 24 = 144$ 7. $2 \cdot ((3 \cdot 4) \cdot (5 + 1)) = 2 \cdot (12 \cdot 6) = 2 \cdot 72 = 144$ 8. $((2 \cdot 3) \cdot 4) \cdot (5 + 1) = (6 \cdot 4) \cdot 6 = 24 \cdot 6 = 144$ 9. $(2 \cdot (3 \cdot 4)) \cdot (5 + 1) = (2 \cdot 12) \cdot 6 = 24 \cdot 6 = 144$
From the evaluations, we see that the possible values are 121, 122, 126, and 144. Therefore, there are 4 distinct values that can be obtained.
The final answer is $\boxed{4}$.

Figure 7 Full question and the correct response generated by the SePT model, corresponding to the analysis in Figure 2. The soft blue and light gray backgrounds distinguish the initial reasoning prefix ($\hat{A}$) from the subsequent correct suffix ($\hat{B}$). This segmentation is because $\hat{B}$ contains the core reasoning and calculation leading to the answer, while $\hat{A}$ represents the initial problem setup.

F Missing Mathematical Derivations

F.1 Score-Function Identity

The following derivation is the standard score-function identity, which shows that one step update of SePT with $\tau_s = \tau_t = \tau$ is equivalent to a random gradient noise update.

$$\begin{aligned}
\mathbb{E}_{o\sim\pi_\theta(\cdot|q)}\left[\nabla_\theta \log \pi_\theta(o \mid q)\right] &= \sum_o \pi_\theta(o \mid q) \cdot \frac{\nabla_\theta \pi_\theta(o \mid q)}{\pi_\theta(o \mid q)} \\
&= \sum_o \nabla_\theta \pi_\theta(o \mid q) \\
&= \nabla_\theta \sum_o \pi_\theta(o \mid q) \\
&= \nabla_\theta 1 \\
&= 0.
\end{aligned} \tag{5}$$

F.2 Proofs for Proposition 1 and Theorem 1

We work within a single SePT round, so that the low-temperature sampling policy

$$b(a \mid \zeta) := \pi_{\text{old}}(a \mid \zeta; \tau_s)$$

is fixed throughout the round. Let $\mathcal{D}$ denote the prompt distribution. For a sampled rollout $o = (o_1, \ldots, o_{|o|}) \sim b(\cdot \mid q)$, define the prefix state at step k by

$$\zeta_k := (q, o_{<k}).$$

Also define the training policy

$$p_\theta(a \mid \zeta) := \pi_\theta(a \mid \zeta; \tau_t).$$

We will use b and p_θ to denote the teacher and student models throughout the proof to ease the notation. With these definitions, we are ready to prove Proposition 1 and Theorem 1.

Proof of Proposition 1. By autoregressive factorization,

$$-\log \pi_\theta(o \mid q; \tau_t) = -\sum_{k=1}^{|o|} \log p_\theta(o_k \mid \zeta_k).$$

Let $\mathbb{P}_b$ denote the joint law of

$$q \sim \mathcal{D},\ o \sim b(\cdot \mid q),$$

and let $\mathbb{E}_b$ denote be expectation under that law. Therefore,

$$\mathcal{L}_{\text{SePT}}(\theta) = \mathbb{E}_b\left[\sum_{k=1}^{|o|} -\log p_\theta(o_k \mid \zeta_k)\right].$$

Grouping terms by prefix state gives

$$\mathcal{L}_{\text{SePT}}(\theta) = \sum_\zeta \mathbb{E}_b\left[\sum_{k=1}^{|o|} \mathbf{1}\{\zeta_k = \zeta\}\big(-\log p_\theta(o_k \mid \zeta)\big)\right].$$

Conditioned on $\zeta_k = \zeta$, the next token o_k is distributed as $b(\cdot \mid \zeta)$. Hence, taking the conditional expectation with respect to the distribution of the next token o_k, we have

$$\mathbb{E}_b[-\log p_\theta(o_k \mid \zeta) \mid \zeta_k = \zeta] = \sum_a b(a \mid \zeta)\big(-\log p_\theta(a \mid \zeta)\big).$$

Define the prefix-occupancy measure

$$d_{\text{old}}(\zeta) := \mathbb{E}_b\left[\sum_{k=1}^{|o|} \mathbf{1}\{\zeta_k = \zeta\}\right].$$

Substituting the conditional expectation above into the loss function yields

$$\begin{aligned}
\mathcal{L}_{\text{SePT}}(\theta) &= \sum_{\zeta}\sum_{k} \mathbb{E}_b\big[\mathbf{1}\{\zeta_k = \zeta\}\,\big(-\log p_\theta(o_k \mid \zeta_k)\big)\big] \\
&= \sum_{\zeta}\sum_{k} \mathbb{P}_b(\zeta_k = \zeta)\,\mathbb{E}_b[-\log p_\theta(o_k \mid \zeta_k) \,|\, \zeta_k = \zeta] \\
&= \sum_{\zeta}\sum_{k} \mathbb{P}_b(\zeta_k = \zeta)\,\sum_{a} b(a \mid \zeta)\big(-\log p_\theta(a \mid \zeta)\big) \\
&= \sum_{\zeta} d_{\text{old}}(\zeta)\,\sum_{a} b(a \mid \zeta)\big(-\log p_\theta(a \mid \zeta)\big).
\end{aligned}$$

Using the well-known identity (i.e., cross-entropy decomposition)

$$\sum_{a} b(a \mid \zeta)\big(-\log p_\theta(a \mid \zeta)\big) = H\big(b(\cdot \mid \zeta)\big) + \mathrm{KL}\big(b(\cdot \mid \zeta)\,\|\,p_\theta(\cdot \mid \zeta)\big)$$

with H being entropy, we obtain

$$\mathcal{L}_{\text{SePT}}(\theta) = \sum_{\zeta} d_{\text{old}}(\zeta)\,H\big(b(\cdot \mid \zeta)\big) + \sum_{\zeta} d_{\text{old}}(\zeta)\,\mathrm{KL}\big(b(\cdot \mid \zeta)\,\|\,p_\theta(\cdot \mid \zeta)\big).$$

Setting

$$C_b := \sum_{\zeta} d_{\text{old}}(\zeta)\,H\big(b(\cdot \mid \zeta)\big)$$

completes the proof. Since b is fixed within the round, C_b is independent of θ. □

Proof of Theorem 1. By Proposition 1, for any prefix ζ with $d_{\text{old}}(\zeta) > 0$, the local contribution to the objective is

$$\sum_{a} b(a \mid \zeta)\big(-\log p_\theta(a \mid \zeta)\big) = H\big(b(\cdot \mid \zeta)\big) + \mathrm{KL}\big(b(\cdot \mid \zeta)\,\|\,p_\theta(\cdot \mid \zeta)\big).$$

Note that we have omitted $d_{\text{old}}(\zeta)$ in this local loss as it is a constant and is positive. Since the entropy term is constant with respect to $p_\theta(\cdot \mid \zeta)$, while the KL term is nonnegative and equals zero if and only if

$$p_\theta(\cdot \mid \zeta) = b(\cdot \mid \zeta).$$

Hence, the pointwise optimum satisfies

$$p^*_\theta(\cdot \mid \zeta) = b(\cdot \mid \zeta),$$

as the lower bound zero is attained at this solution.

Now write

$$b(\cdot \mid \zeta) = \mathrm{softmax}\left(\frac{z_{\text{old}}(\zeta)}{\tau_s}\right), \qquad p^*_\theta(\cdot \mid \zeta) = \mathrm{softmax}\left(\frac{z^*(\zeta)}{\tau_t}\right).$$

Since $p^*_\theta(\cdot \mid \zeta) = b(\cdot \mid \zeta)$, we have

$$\mathrm{softmax}\left(\frac{z^*(\zeta)}{\tau_t}\right) = \mathrm{softmax}\left(\frac{z_{\text{old}}(\zeta)}{\tau_s}\right).$$

A standard property of the softmax function is that

$$\mathrm{softmax}(u) = \mathrm{softmax}(v) \quad \Longleftrightarrow \quad u = v + c\mathbf{1}$$

for some scalar c, as softmax is invariant to a constant additive shift. Therefore, there exists $c(\zeta) \in \mathbb{R}$ such that

$$\frac{z^*(\zeta)}{\tau_t} = \frac{z_{\text{old}}(\zeta)}{\tau_s} + c(\zeta)\mathbf{1},$$

which is equivalent to

$$z^*(\zeta) = \frac{\tau_t}{\tau_s} z_{\text{old}}(\zeta) + c(\zeta)\mathbf{1},$$

after absorbing τ_t into the scalar $c(\zeta)$. Subtracting the j-th coordinate from the i-th coordinate gives

$$z_i^*(\zeta) - z_j^*(\zeta) = \frac{\tau_t}{\tau_s}\big(z_i^{\text{old}}(\zeta) - z_j^{\text{old}}(\zeta)\big).$$

Therefore, when $\tau_s < \tau_t$, every pairwise logit margin is amplified by the factor $\tau_t/\tau_s > 1$. □

G Background on SFT and GRPO

We provide backgrounds on the training formulations for SFT and GRPO.

G.1 Supervised Finetuning (SFT)

SFT is a standard technique for adapting a pre-trained model π_θ to specific tasks by training it on a static dataset $\mathcal{D}$ of high-quality prompt-response pairs (q, o). The training objective of SFT is formulated by minimizing the negative log-likelihood loss over $\mathcal{D}$:

$$\mathcal{L}_{\text{SFT}}(\theta; \mathcal{D}) = -\mathbb{E}_{(q,o)\sim\mathcal{D}}\left[\sum_{k=1}^{|o|} \log \pi_\theta(o_k \mid q, o_{<k})\right]. \tag{6}$$

SFT is commonly applied for adapting a pre-trained model to follow human instructions or to manage specific downstream tasks. In terms of incentivizing the model's reasoning ability, one can also perform SFT on long CoT data distilled from strong reasoning models, where the response o contains rich reasoning traces; see, e.g., [22–26].

G.2 Reinforcement Learning with Verifiable Rewards

For reasoning tasks where solutions can be programmatically verified, on-policy RL is a popular approach for improving the model's reasoning ability. A popular method in this domain is Group Relative Policy Optimization (GRPO) [8]. GRPO is to maximize the following clipped surrogate objective derived from Proximal Policy Optimization (PPO):

$$\begin{aligned} J(\theta) =& \mathbb{E}_{q\sim\mathcal{D}, o\sim\pi_{\text{old}}(\cdot|q)}\Bigg[\sum\nolimits_{k=1}^{|o|} \min\left(\frac{\pi_\theta(o_k|q,o_{<k})}{\pi_{\text{old}}(o_k|q,o_{<k})}\hat{A}(q,o),\right. \\ & \left.\text{clip}\left(\frac{\pi_\theta(o_k|q,o_{<k})}{\pi_{\text{old}}(o_k|q,o_{<k})}, 1-\epsilon, 1+\epsilon\right)\hat{A}(q,o)\right)\Bigg] - \beta\,\mathbb{E}_{q\sim\mathcal{D}}\Big[D_{\text{KL}}\big(\pi_\theta(\cdot|q)\|\pi_{\text{ref}}(\cdot|q)\big)\Big], \end{aligned} \tag{7}$$

where $\hat{A}(q,o) = \frac{r(q,o)-\bar{r}_q}{\sigma_q}$ is the advantage with r being reward and D_{KL} is the KL divergence.